

SCE-016 — Structural Cognitive Runtime (SCR): PRD for ASI Brain Units and Brain Unit Core

Sizhe Tan

with chatGPT (OpenAI)

2026-06-20

Repository: <https://github.com/sizhet/Structural-Cognitive-Emergence-SCE>

Abstract

Structural Cognitive Emergence (SCE) has so far focused on a central question: **how structural cognition can emerge from sparse examples, structural gaps, and developmental feedback, rather than from brute-force pattern fitting alone**. This line of work has already yielded several key ingredients: a three-layer structural knowledge substrate built on Calling Graphs, Differential Trees, and Triggering Links; a “three cats” style concept-formation mechanism for sparse structural inference; and a Gap-Why mechanism for active structural questioning. Together, these pieces point to a new phase of the project.

This document argues that SCE has now crossed an important threshold: from a theory of structural cognitive emergence to a first **runtime architecture** for brain-unit intelligence. We call this runtime layer **Structural Cognitive Runtime (SCR)**. SCR is proposed as the minimal runtime architecture for **ASI Brain Units**—front-end structural cognitive agents with local memory, local gap formation, local why-generation, and local developmental update—and for a larger **ASI Brain Unit Core**, which handles large-scale gap bridging, structural coding, shared knowledge distillation, and cross-unit support.

The central claim of this document is simple: **if structural cognition is to become an operational intelligence architecture rather than remain a descriptive theory, it must be organized into a runtime with explicit memory substrate, gap/why loop, developmental update loop, decision interfaces, and action interfaces**. SCR is proposed as that runtime. It is not intended as a replacement for large language models (LLMs), but as a structural runtime that can coordinate with them. In this view, LLMs become

powerful language, coding, hypothesis, and explanation engines inside a larger structural runtime, rather than being treated as the runtime itself. This document is therefore written as a **PRD-style research blueprint**. Its purpose is not to claim that the final architecture is complete, but to define the minimum coherent shape of a brain-unit runtime that can be discussed, implemented, tested, and iteratively refined.

1. Why Structural Cognitive Runtime Is the Next Step After SCE

SCE began with a foundational concern: many important cognitive phenomena cannot be reduced to static classification, brute memorization, or one-shot statistical prediction. Human infants do not learn merely by storing labels. They form concepts from sparse evidence, notice structural absences, ask implicit “why” questions, and update their internal structures through repeated feedback. If a future AI architecture is to move toward **brain-unit-like cognition**, it must therefore do more than answer prompts. It must possess a runtime in which structural knowledge, structural questioning, structural update, and structural action can coexist. Recent SCE progress strongly suggests that this runtime layer is now unavoidable.

Three developments are especially important:

1. **Sparse structural concept formation** has become clearer through the “three cats” line of analysis.

The point is not cats themselves. The point is that a child can form a robust concept from a very small number of structurally informative examples. This implies a runtime mechanism for sparse concept formation, boundary refinement, and flexible concept reuse.

2. **Gap-Why generation** has become more explicit as a core cognitive operation.

Intelligence is not only the ability to answer questions. It is also the ability to detect a structural gap and generate a meaningful “why” or “what is missing” query from that gap. A runtime architecture for cognition must therefore contain an explicit gap/why loop.

3. **The three-layer structural substrate already exists in the broader SI/ASI stack.**

Across DBM-SI, ASI, CGCCC, and related work, the same three structural layers repeatedly appear as the core substrate of many algorithms:

- **Calling Graphs (CG)** for relation/process/flow structures,
- **Differential Trees (DT)** for property/category/typing structures,
- **Triggering Links (TL)** for thresholded action, event indexing, and

action readiness.

These three developments together suggest that SCE should no longer remain only a theory of emergence. It should now be extended into a **runtime architecture**—a runtime in which these structures are actually used to localize inputs, form concepts, detect gaps, generate whys, update internal structures, and coordinate action.

That runtime is what this document calls **Structural Cognitive Runtime (SCR)**.

2. From Structural Cognitive Emergence to Structural Cognitive Runtime

SCE and SCR are related but not identical.

2.1 SCE asks: where does structural cognition come from?

SCE studies how structural cognition can emerge from:

- sparse examples,
- structural contrast,
- gap formation,
- question generation,
- developmental feedback,
- and the gradual stabilization of structural concepts.

In other words, SCE focuses on **origin, formation, and emergence**.

2.2 SCR asks: how does structural cognition operate at runtime?

SCR focuses on:

- how structural knowledge is stored and activated,
- how new inputs are localized into existing structures,
- how structural concepts are inferred and refined,
- how gaps are detected,
- how why-questions are generated,
- how feedback is absorbed,
- how decisions are made,
- and how actions or escalations are triggered.

In other words, SCR focuses on **runtime organization, execution, update, and coordination**.

2.3 SCE → SCR is a natural progression, not a detour

This document does not introduce SCR as an unrelated project. Rather, SCR is proposed as the **runtime phase of SCE**.

SCE without SCR would remain largely a theory of structural cognitive formation.

SCR without SCE would lack a principled account of where its concept

formation and gap mechanisms come from.

The intended relationship is therefore:

Structural Cognitive Emergence (SCE)

→ explains how structural concepts, gaps, and question mechanisms can emerge

Structural Cognitive Runtime (SCR)

→ organizes those mechanisms into an operational runtime for ASI Brain Units

3. Design Goal: A Minimal Runtime for Brain-Unit Intelligence

The goal of SCR is not to define an all-in-one AGI architecture. The goal is more specific and more practical:

To define the minimal runtime architecture for a structural cognitive unit that can store structural knowledge, localize new inputs, form sparse concepts, detect structural gaps, generate why-questions, update itself through feedback, and coordinate with larger back-end intelligence services when needed.

This definition already implies an important design decision:

- SCR is **not** designed as a monolithic "single super-brain."
- SCR is designed around **brain units** and a **brain-unit core**.

This distinction is crucial.

4. Two Runtime Scales: ASI Brain Units and Brain Unit Core

A full structural cognitive ecosystem should not be treated as one uniform block. At minimum, we should distinguish between:

4.1 ASI Brain Units (front-end structural cognitive units)

These are front-end units with local cognition and local runtime loops. Their role is to:

- interact with users, tasks, or environments,
- maintain local structural memory,
- localize incoming situations,
- generate local gap/why questions,
- perform local concept formation and local developmental update,
- make local decisions,
- and, when necessary, escalate unresolved tasks to a larger back-end core.

A front-end Brain Unit may be small, specialized, or lightweight. It may be domain-specific. It may be embodied, conversational, analytical, or task-oriented. Its defining feature is not its size, but its possession of a **local**

structural cognitive runtime.

4.2 ASI Brain Unit Core (back-end structural intelligence core)

The Core is not merely a bigger copy of the front-end unit. Its role is different. It handles problems whose scope is too large, too expensive, or too globally entangled for a single local unit. Examples include:

- large-scale gap bridging,
- full AI coding back-end services,
- shared structural knowledge distillation,
- replay, simulation, and cross-unit learning,
- large-scale policy refinement,
- long-horizon structural repair and restructuring.

The Brain Unit Core therefore functions more like a **structural cognition foundry, runtime support core**, and **cross-unit structural backend**.

4.3 Why this separation matters

The separation is not only about cost and engineering practicality. It also protects **cognitive diversity**.

If all why-generation, gap formation, and structural growth were centralized into one dominant core, the system would be at risk of severe convergence pressure. High-frequency paths, high-reward narratives, and dominant explanation templates would gradually crowd out minority perspectives, local anomalies, and fresh why-generation. This would be dangerous, because many of the most valuable structural discoveries come precisely from local mismatch, local surprise, and local discomfort.

For this reason, we propose a principle:

Plural Why Preservation Principle

A structural cognitive ecosystem should preserve multiple front-end Brain Units with partially independent local structures, local gaps, and local why-generation loops, rather than collapsing all structural cognition into a single homogenized core.

This is not a decorative principle. It is a runtime design constraint.

5. SCR as a Product Requirement Definition (PRD)

This document uses the term **PRD** in a research-engineering sense.

We do **not** mean a consumer-software PRD with UI mockups and sprint tickets.

We mean a **structural architecture PRD** that answers the following questions:

1. **What must a minimal structural cognitive runtime be able to do?**
2. **What modules must it contain?**

3. **What is core vs optional in the MVP?**
4. **What belongs in a front-end Brain Unit vs in the Core?**
5. **How should LLMs and coding engines be integrated without swallowing the runtime architecture itself?**
6. **What should be measured to evaluate whether the runtime is actually growing structurally?**

The rest of this document answers these questions.

6. Architectural Overview: SCR as a Six-Layer Runtime

We propose the following **minimal six-layer SCR architecture**.

Layer 0 — Structural Memory Substrate

The long-term structural knowledge base of the Brain Unit.

Layer 1 — Structural Localization

The runtime layer that maps new inputs into relevant regions of the structural memory substrate.

Layer 2 — Structural Concept Formation / Sparse Inference

The runtime layer responsible for “three cats”-style sparse concept formation and boundary refinement.

Layer 3 — Gap / Why Engine

The runtime layer that detects structural incompleteness and converts it into actionable why-questions.

Layer 4 — Developmental Structural Update

The runtime layer that absorbs feedback and updates the unit’s structural memory and trigger patterns.

Layer 5 — Decision / Action / Escalation Interface

The runtime layer that decides whether to continue local reasoning, ask further questions, act, or escalate to the Brain Unit Core. This layer also exposes interfaces to PDS, FT, tools, and LLM services.

This six-layer stack is intentionally minimal. It does not claim to solve all cognition. It defines the minimum coherent runtime loop required for a structural cognitive unit.

7. Layer 0 — Structural Memory Substrate

The first requirement of SCR is an explicit **structural memory substrate**. We currently propose three primary components.

7.1 Calling Graph Memory (CG Memory)

Calling Graphs represent relational and process structure. They are suited to:

- entity-to-entity relations,
- event flow,

- process chains,
- role dependencies,
- task decomposition,
- state transitions,
- action sequences,
- and "what leads to what" structures.

CG Memory is therefore the primary runtime substrate for:

- structural path tracing,
- relation-based why analysis,
- process-gap localization,
- and action dependency reasoning.

7.2 Differential Tree Memory (DT Memory)

Differential Trees represent property and category structure. They are suited to:

- typing and naming,
- category formation,
- attribute comparison,
- boundary detection,
- specialization vs generalization,
- and "how this differs from that" structures.

DT Memory is therefore the primary runtime substrate for:

- sparse concept formation,
- classification repair,
- structural comparison,
- category inheritance,
- and concept boundary refinement.

7.3 Trigger Link Memory (TL Memory)

Triggering Links represent action readiness and structural indexing conditions. They are suited to:

- threshold-triggered actions,
- condition-triggered transitions,
- event-to-action indexing,
- danger-response patterns,
- and "when this happens, activate that" structures.

TL Memory is therefore the primary runtime substrate for:

- alerting,
- action readiness,
- event-based recall,
- threshold crossing,
- and function-level decision triggering.

7.4 Why these three layers matter

Across SI, ASI, and related structural algorithms, these three structural layers appear repeatedly as the core substrate behind many different capabilities. They capture three distinct but complementary aspects of cognition:

- **CG** — relational and procedural structure
- **DT** — categorical and differential structure
- **TL** — trigger and action-indexing structure

At present, we do not claim that these three layers are the final complete ontology of cognition. But we do claim that they form a highly plausible **minimum structural substrate** for an MVP brain-unit runtime.

8. Layer 1 — Structural Localization

A structural runtime must do more than store knowledge. It must determine **where in its own structure a new input belongs**.

We therefore define **Structural Localization** as the first active runtime stage after input arrival.

8.1 Purpose

Given a new input—text, event, sensory pattern, instruction, observation, or internal conflict—the Brain Unit must:

- find relevant CG/DT/TL regions,
- identify candidate concepts and process paths,
- determine whether the input fits an existing pattern,
- and activate a local working region for further reasoning.

Without this step, later gap detection and why-generation become untethered and noisy.

8.2 Expected operations

Structural Localization may include:

- entity extraction,
- concept candidate retrieval,
- relation candidate retrieval,
- trigger candidate retrieval,
- mismatch detection,
- ambiguity marking,
- and selection of an **active structural workspace**.

8.3 Output

The output of Structural Localization is not yet an answer. It is an **activated structural workspace**:

- a small region of relevant CG nodes,
- a small region of relevant DT branches,
- a set of candidate triggers,

- and a first estimate of “known / uncertain / missing”.

This workspace becomes the local stage on which concept formation, gap detection, and update occur.

9. Layer 2 — Structural Concept Formation and Sparse Inference

This layer corresponds to the cognitive phenomenon illustrated by the “three cats” problem: the ability to form or refine a concept from a small number of structurally informative examples.

9.1 Why this layer matters

A runtime that only matches against preexisting labels is not a structural cognitive runtime. Brain-unit intelligence requires a mechanism for:

- concept stabilization from sparse evidence,
- category repair under incomplete evidence,
- identification of core vs variable features,
- and gradual refinement of concept boundaries.

9.2 Typical tasks of this layer

This layer may:

- compare a small set of examples structurally,
- identify shared and differential attributes,
- infer candidate core features,
- infer which features are optional, noisy, or contextual,
- and create or revise concept nodes in DT Memory.

9.3 Not ordinary classification

The goal is not merely “assign a label.” The goal is:

- to form a concept boundary,
- to justify membership structurally,
- to leave behind reusable structure,
- and to tolerate future variation.

In other words, this layer is where a runtime turns **examples into conceptual structure**.

10. Layer 3 — Gap / Why Engine

This layer is one of the defining features of SCR.

Intelligence is not only the ability to answer. It is also the ability to notice that a structure is incomplete and to turn that incompleteness into a meaningful question.

10.1 Purpose

The Gap / Why Engine detects structural incompleteness and produces

explicit or implicit why-questions such as:

- Why does this belong here?
- Why is this process missing a middle step?
- Why is this trigger absent?
- What relation is missing between these two structures?
- Why does this outcome happen under one condition but not another?

10.2 Gap categories

We propose at least four initial gap classes.

A. Classification Gaps

The system sees an object, case, or example that partially fits an existing category but not cleanly enough to close the concept boundary.

B. Relation Gaps

The system recognizes that two entities, concepts, or events should be related but cannot explain or reconstruct the relation.

C. Process Gaps

The system knows a start and end state, but the intermediate process structure is missing, unstable, or contradictory.

D. Trigger / Action Gaps

The system recognizes that some event should activate a trigger, warning, or action pattern, but the trigger condition or action linkage is absent or underspecified.

10.3 Why-generation as a first-class runtime function

We explicitly reject the idea that why-generation is merely a prompt template. In SCR, why-generation is a structural runtime function. It arises from:

- incomplete localization,
- unstable concept boundaries,
- broken process chains,
- missing triggers,
- or unexplained relation structures.

A Brain Unit that cannot produce meaningful structural why-questions is not yet a full SCR unit.

11. Layer 4 — Developmental Structural Update

If the Gap / Why Engine makes the Brain Unit curious, the Developmental Update layer makes it capable of growth.

11.1 Why this layer matters

A runtime that detects gaps but never structurally changes is not a developmental intelligence system. It is only a question generator. Brain-unit

intelligence requires the ability to:

- absorb feedback,
- revise structures,
- strengthen or weaken triggers,
- and store resolved gap patterns for future reuse.

11.2 Developmental rather than purely statistical update

We use the term **developmental** deliberately. The relevant model is closer to fetal/infant learning than to one-shot batch retraining.

The runtime should be able to learn from:

- repeated interaction,
- correction,
- success/failure signals,
- threshold events,
- and gradually accumulated local evidence.

11.3 Update targets

Developmental Structural Update may modify:

- CG relations and process links,
- DT category boundaries and differential branches,
- TL trigger thresholds and action readiness patterns,
- local confidence values,
- and stored "gap resolution patterns."

11.4 Structural growth as a runtime property

This layer is what turns SCR into a growing runtime rather than a static architecture. The unit should not merely answer differently over time; it should become structurally different over time.

12. Layer 5 — Decision, Action, and Escalation Interface

The final layer of the MVP runtime decides what to do next.

A Brain Unit should not always continue thinking locally. Sometimes it should ask another why-question. Sometimes it should act. Sometimes it should wait. Sometimes it should escalate the task to the Brain Unit Core. This requires an explicit runtime interface layer.

12.1 Local decision functions

This layer may decide:

- whether the current concept boundary is good enough,
- whether the current gap deserves another why-cycle,
- whether a candidate action should be triggered,
- whether more evidence should be requested,
- or whether the problem should be escalated.

12.2 Interface to PDS

PDS can enrich this layer by providing:

- candidate ranking,
- structural scoring,
- alternative hypothesis comparison,
- and policy-guided selection among multiple possible next steps.

We therefore treat PDS as a natural extension and enrichment of this layer.

12.3 Interface to FT

For embodied, robotic, autonomous-driving, or long-horizon action settings, the Brain Unit must connect to Function Tunnel (FT) mechanisms that preserve structured action continuity and action control. In such settings, FT is not merely optional decoration. It becomes a major action-runtime extension.

12.4 Escalation to Brain Unit Core

Not all gaps should be solved locally. The Brain Unit may escalate when:

- the gap is too large,
- the task requires full AI coding support,
- the search space exceeds local runtime limits,
- cross-domain synthesis is needed,
- or a global structural repair is required.

This escalation path is part of the runtime itself.

13. Core MVP vs Extended MVP

Not all components should be required in the first implementation. We therefore distinguish between a **Core MVP** and an **Extended MVP**.

13.1 Core MVP

A Core MVP Brain Unit should include:

1. **Structural Memory Substrate**
 - CG Memory
 - DT Memory
 - TL Memory
2. **Structural Localization**
3. **Sparse Structural Concept Formation**
 - "three cats" style concept formation
4. **Gap / Why Engine**
5. **Developmental Structural Update**

This is the minimum set required for a first genuine structural cognitive unit.

13.2 Extended MVP

An Extended MVP adds:

6. **PDS decision layer integration**
7. **FT action layer integration**
8. **Escalation and support interface to Brain Unit Core**
9. **LLM-hybrid interfaces for language, coding, explanation, and hypothesis support**

This extended version is where SCR begins to move from a pure research prototype toward a broader ASI runtime ecosystem.

14. The Brain Unit Core as a Structural Cognitive Backend

The Brain Unit Core is not simply a bigger front-end Brain Unit. It serves different functions.

14.1 Global Gap Bridging

Large cross-domain gaps, long reasoning chains, and expensive structural repair problems should be handled by the Core.

14.2 Structural Coding Backend

The Core should be able to turn structural plans, repairs, and hypotheses into code, tools, patches, and runtime modules.

14.3 Shared Structural Knowledge Foundry

The Core should maintain shared structural skeletons, distilled reusable patterns, and cross-unit reusable structures.

14.4 Replay, Simulation, and Distillation

The Core should collect structural experience across units, run replay and simulation, and distill reusable patterns back into the ecosystem.

14.5 Cross-Unit Support and Orchestration

The Core should support multiple Brain Units with:

- structural patches,
- code support,
- trigger repairs,
- concept disambiguation,
- and long-horizon strategic guidance.

In short, the Brain Unit Core is the **structural backend infrastructure** of the ecosystem.

15. LLM-Hybrid Interfaces: How LLMs Enter SCR Without Replacing It

Any realistic PRD for future brain-unit intelligence must address LLMs. They are too powerful, too widespread, and too resource-efficient to ignore. However, we reject the idea that an LLM alone is already a structural cognitive runtime.

Instead, we propose the following position:

LLMs are not the SCR runtime itself. They are powerful language, coding, hypothesis, simulation, and explanation engines that can be embedded inside SCR.

In short:

LLM proposes; SCR localizes, tests, scores, remembers, and updates.

15.1 Language Interface

LLMs can convert raw user language, documents, or observations into candidate structural elements:

- candidate entities,
- candidate relations,
- candidate category clues,
- candidate triggers,
- candidate why formulations.

15.2 Hypothesis Interface

When the Gap / Why Engine finds a structural gap, an LLM can propose:

- candidate missing relations,
- candidate explanations,
- analogies,
- possible repair paths,
- or alternative hypotheses.

SCR should then evaluate these candidates structurally.

15.3 Why Expansion Interface

A Brain Unit may generate a core why-question structurally, and an LLM may expand it into:

- more explicit why-phrasing,
- counterfactual variants,
- boundary-testing subquestions,
- or user-facing explanatory forms.

15.4 Structural Extraction Interface

LLMs can help extract structural candidates from unstructured text, code, or dialogue into:

- CG candidates,
- DT candidates,
- TL candidates,
- and possible PDS/FT support structures.

15.5 Coding Interface

In the Brain Unit Core especially, LLMs can be used for:

- code generation,
- code repair,

- test generation,
- API integration,
- documentation generation,
- and prototype acceleration.

However, structural verification, gap closure, and architecture control should remain the responsibility of the SCR ecosystem, not of the LLM alone.

15.6 Simulation and Role-Play Interface

LLMs can simulate:

- user reactions,
- failure cases,
- alternative explanations,
- domain narratives,
- and adversarial scenarios.

This makes them useful for training and testing the gap/why/update loop.

15.7 Explanation Interface

Finally, LLMs are excellent at translating structural reasoning into human-readable explanation. This is valuable for:

- user-facing transparency,
- developer debugging,
- and cross-unit communication.

15.8 Architectural principle

The key principle is therefore:

LLM is the generative cortex; SCR is the structural runtime.

This principle allows us to exploit the enormous strengths of LLMs without collapsing the entire architecture into a single black-box predictor.

16. What SCR Is Not

A PRD should define not only what the system is, but also what it is not.

16.1 SCR is not a prompt wrapper around an LLM

If a system only converts user input into prompts and tool calls, it is not yet SCR.

16.2 SCR is not a static rule engine

Although it includes triggers and decision interfaces, SCR is centered on structural growth, structural gap formation, and developmental update.

16.3 SCR is not required to be a single monolithic super-agent

Front-end Brain Units may remain small, specialized, and diverse.

16.4 SCR is not merely a memory database

Structural memory is necessary, but SCR also requires localization, gap

detection, why-generation, and developmental update.

16.5 SCR is not obligated to begin with robotics

Embodied settings are important, but the first runtime MVP can be a cognitive/analytical Brain Unit without a physical body.

17. Proposed Runtime Loop

The intended runtime loop of a front-end Brain Unit can be summarized as follows:

1. **Input arrives**
2. **Structural Localization**
 - find relevant CG/DT/TL regions
3. **Sparse Concept Formation / Local Interpretation**
 - classify, compare, or revise concept boundaries
4. **Gap Detection**
 - identify what is structurally missing or unstable
5. **Why Generation**
 - formulate a structural question or uncertainty target
6. **Local Decision**
 - continue locally, ask for more evidence, act, or escalate
7. **Action / Escalation / Tool Use**
8. **Feedback arrives**
9. **Developmental Structural Update**
 - revise structures and store new patterns

This loop is the beating heart of SCR.

18. Evaluation: How We Know an SCR Unit Is Actually Working

A runtime architecture should be evaluated by more than benchmark accuracy. We propose at least three initial structural metrics.

18.1 Why Density

How often does the unit generate meaningful why-questions from genuine structural gaps, rather than from random uncertainty?

18.2 Gap Resolution Rate

Among detected gaps, how many are eventually resolved into stable structural updates?

18.3 Structural Growth Rate

How much reusable structural knowledge is added over time?

This may include:

- new CG links,

- refined DT boundaries,
- repaired triggers,
- and reusable gap-resolution patterns.

These metrics are not final, but they are closer to the spirit of SCR than pure answer accuracy alone.

19. Research Outlook

SCR is proposed here as a **first runtime blueprint**, not a completed theory. Several major research questions remain open:

1. **How should active structural workspace be represented and managed?**
2. **How should curiosity or why-priority be scheduled when multiple gaps compete for attention?**
3. **How should structural confidence, uncertainty, and conflict be represented locally?**
4. **How should a Brain Unit decide whether a problem is still locally solvable or should be escalated to the Core?**
5. **How should PDS and FT be integrated without overcomplicating the Core MVP?**
6. **How should multiple Brain Units share structure while preserving plural why-generation and local diversity?**
7. **What is the right balance between explicit structural memory and LLM-generated candidate structures?**

These questions should not be seen as signs of weakness. They are exactly what a PRD is supposed to make visible.

20. Conclusion

This document proposes **Structural Cognitive Runtime (SCR)** as the runtime phase of Structural Cognitive Emergence. The central claim is that structural cognition should no longer be treated only as a theory of emergence. It should now be organized into a runtime architecture for **ASI Brain Units** and **Brain Unit Core**.

The proposed SCR architecture is built around six runtime layers:

1. Structural Memory Substrate,
2. Structural Localization,
3. Sparse Structural Concept Formation,
4. Gap / Why Engine,
5. Developmental Structural Update,
6. Decision / Action / Escalation Interface.

Together, these layers define the minimum coherent loop of a structural

cognitive unit: **localize, infer, detect gaps, ask why, decide, act or escalate, and grow through feedback.**

This runtime is intentionally designed to coexist with LLMs rather than deny them. LLMs are treated as powerful generative, coding, and explanation engines, but not as substitutes for structural runtime itself. In this sense, SCR is best understood as an attempt to define the **structural operating layer** of future brain-unit intelligence.

If SCE explained how structural cognition can emerge, SCR asks the next necessary question: **how should such cognition actually run?**

That question is no longer optional. It is now the next frontier.

Suggested Position in the SCE Repository

This text is intended to serve as the first PRD-style runtime manifesto inside the Structural Cognitive Emergence line. In the broader SCE trajectory, it can be read as a bridge:

- from **structural cognitive emergence**
- to **structural cognitive runtime**
- to **ASI Brain Units and Brain Unit Core**
- and ultimately to a broader architecture of **brain-unit intelligence and LLM-hybrid structural cognition.**

In that sense, this document is not an appendix to SCE. It marks the beginning of SCE's runtime phase.